

To unlock these creative features, replace your core server files with the updated logic below. These additions implement flexible listing filters, computational math tracking, system snapshots, and highlight the limits of temporary server storage.

Node.js Lane (Express)

Update your `server.js` file to support query filtering, a status summary, and seed mechanics:

javascript

```
const express = require('express');
const app = express();
const PORT = 3000;

app.use(express.json());

// Default seed list data
const initialTasks = [
  { id: 1, title: "Buy groceries", done: false },
  { id: 2, title: "Clean the room", done: true },
  { id: 3, title: "Study API design", done: false }
];

// In-memory runtime database array
let tasks = [...initialTasks];

app.get('/', (req, res) => {
  res.json({ name: "Task API", version: "1.0", endpoints: ["/tasks", "/stats"] });
});

app.get('/health', (req, res) => {
  res.json({ status: "ok" });
});

// GET /tasks - Supports ?done=true/false and ?search=word parameters
app.get('/tasks', (req, res) => {
  let filteredTasks = [...tasks];
  const { done, search } = req.query;

  // 1. Filter by completed status flag
  if (done !== undefined) {
```

```
    const isDone = done === 'true';
    filteredTasks = filteredTasks.filter(t => t.done === isDone);
  }

  // 2. Filter by matching title keyword text
  if (search !== undefined && search.trim() !== "") {
    filteredTasks = filteredTasks.filter(t =>
      t.title.toLowerCase().includes(search.toLowerCase())
    );
  }

  res.json(filteredTasks);
});

// GET /stats - Calculates summary information
app.get('/stats', (req, res) => {
  const total = tasks.length;
  const done = tasks.filter(t => t.done).length;
  const open = total - done;

  res.json({ total, done, open });
});

// POST /reset - Re-seeds initial values
app.post('/reset', (req, res) => {
  tasks = [...initialTasks];
  res.json({ message: "Database reset to initial seed values", count: tasks.length });
});

// Standard CRUD endpoints below
app.get('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const task = tasks.find(t => t.id === taskId);
  if (!task) return res.status(404).json({ error: `Task ${taskId} not found` });
  res.json(task);
});

app.post('/tasks', (req, res) => {
  const { title } = req.body;
  if (!title || title.trim() === "") return res.status(400).json({ error: "Title is required" });
  const nextId = tasks.length > 0 ? Math.max(...tasks.map(t => t.id)) + 1 : 1;
  const newTask = { id: nextId, title, done: false };
  tasks.push(newTask);
  res.status(201).json(newTask);
});
```

```
});

app.put('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const taskIndex = tasks.findIndex(t => t.id === taskId);
  if (taskIndex === -1) return res.status(404).json({ error: `Task ${taskId} not found` });

  const { title, done } = req.body;
  if (title === undefined && done === undefined) return res.status(400).json({ error: "Missing title or done" });
  if (title !== undefined && (!title || title.trim() === "")) return res.status(400).json({ error: "Title is required" });
  if (done !== undefined && typeof done !== "boolean") return res.status(400).json({ error: "Done must be a boolean" });

  if (title !== undefined) tasks[taskIndex].title = title;
  if (done !== undefined) tasks[taskIndex].done = done;
  res.json(tasks[taskIndex]);
});

app.delete('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const taskIndex = tasks.findIndex(t => t.id === taskId);
  if (taskIndex === -1) return res.status(404).json({ error: `Task ${taskId} not found` });
  tasks.splice(taskIndex, 1);
  res.status(204).send();
});

app.listen(PORT, () => console.log(`Server on http://localhost:${PORT}`));
```

Use code with caution.

● Python Lane (FastAPI)

Update your `main.py` file to accept optional string query variables and compute system properties:

python

```
from fastapi import FastAPI, HTTPException, status, Response, Query
from pydantic import BaseModel, Field
from typing import Optional
```

```
app = FastAPI()

class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1)

class TaskUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1)
    done: Optional[bool] = None

INITIAL_TASKS = [
    {"id": 1, "title": "Buy groceries", "done": False},
    {"id": 2, "title": "Clean the room", "done": True},
    {"id": 3, "title": "Study API design", "done": False}
]

tasks = list(INITIAL_TASKS)

@app.get("/")
def read_root():
    return {"name": "Task API", "version": "1.0", "endpoints": ["/tasks", "/stats"]}

@app.get("/health")
def read_health():
    return {"status": "ok"}

# GET /tasks - Dynamic filtering via URL queries
@app.get("/tasks")
def read_tasks(
    done: Optional[bool] = Query(None),
    search: Optional[str] = Query(None)
):
    filtered_tasks = list(tasks)

    if done is not None:
        filtered_tasks = [t for t in filtered_tasks if t["done"] == done]

    if search is not None and search.strip():
        filtered_tasks = [t for t in filtered_tasks if search.lower() in t["title"].lower()]

    return filtered_tasks

# GET /stats - Computational stats engine
@app.get("/stats")
def get_stats():
```

```
total = len(tasks)
done_count = len([t for t in tasks if t["done"]])
open_count = total - done_count
return {"total": total, "done": done_count, "open": open_count}

# POST /reset - Seeds database state
@app.post("/reset")
def reset_database():
    global tasks
    tasks = list(INITIAL_TASKS)
    return {"message": "Database reset to initial seed values", "count": len(tasks)}

# Base CRUD endpoints below
@app.get("/tasks/{id}")
def read_task(id: int):
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})
    return task

@app.post("/tasks", status_code=status.HTTP_201_CREATED)
def create_task(task_input: TaskCreate):
    if not task_input.title.strip():
        raise HTTPException(status_code=400, detail={"error": "Title cannot be empty"})
    next_id = max([t["id"] for t in tasks], default=0) + 1
    new_task = {"id": next_id, "title": task_input.title, "done": False}
    tasks.append(new_task)
    return new_task

@app.put("/tasks/{id}")
def update_task(id: int, task_input: TaskUpdate):
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})
    if task_input.title is None and task_input.done is None:
        raise HTTPException(status_code=400, detail={"error": "Missing fields"})
    if task_input.title is not None:
        if not task_input.title.strip():
            raise HTTPException(status_code=400, detail={"error": "Empty title"})
        task["title"] = task_input.title
    if task_input.done is not None:
        task["done"] = task_input.done
    return task
```

```
@app.delete("/tasks/{id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_task(id: int):
    global tasks
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})
    tasks = [t for t in tasks if t["id"] != id]
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

Use code with caution.

❏ Checkpoint Verification

Fire up your application and try out your new commands with your terminal client
(port or

1. Verify Stats Check:

bash

```
curl -i http://localhost:3000/stats
```

Use code with caution.

2. Verify Title Query Filter:

bash

```
curl -i "http://localhost:3000/tasks?search=room"
```

Use code with caution.

3. Verify Status Flag Filter:

bash

```
curl -i "http://localhost:3000/tasks?done=false"
```

Use code with caution.

4. Verify Wipe and Reset Endpoint:

```
bash
```

```
curl -i -X POST http://localhost:3000/reset
```

Use code with caution.



The Mortality Experiment Documentation

Open up your project `README.md` and append this discovery section to explain how volatile server states work:

```
markdown
```

```
## ⚠️ The Mortality Experiment Reflection
```

```
When new tasks are added to the list and the server process is restarted, all newly create
```

◀

Use code with caution.



Git Commit

Record this extra set of features to your log tree:

```
bash
```

```
git add .  
git commit -m "Extras: query parameters, calculation stats, and reset seeds"  
git push
```